

CRT Sparse Sort (Thanh Ha Algorithm): A Non-Comparative Sorting Framework via the Chinese Remainder Theorem and 2D Coordinate Mapping for Hardware Accelerators

Hao T. Nguyen



Abstract—This paper proposes *CRT Sparse Sort* (Thanh Ha Algorithm), an $O(N)$ linear-time non-comparative sorting method based on transforming 1D integer sets into a 2D coordinate space (S, L) via the Residue Number System (RNS). The proposed framework integrates a Block Tiling technique to support full 64-bit integer ranges, a Mirror Symmetry split mechanism saving 50% of residue storage space, and a 64-bit Bit-Packing Key structure that reduces the sorting process to a single pass over a flat array. The algorithm completely eliminates CPU branch misprediction (Branch-Free Execution), optimizes L1 Cache locality (using a 2 KB Stack lookup buffer), and is particularly suitable for domain-specific hardware architectures (FPGA/ASIC) or cyclic signal processing (DSP/Radar).

Index Terms—Residue Number System, Chinese Remainder Theorem, Non-Comparative Sorting, Bit-Packing, Mirror Symmetry, Block Tiling, Branch-Free Sorting.

1 INTRODUCTION

Sorting algorithms constitute one of the most fundamental operational primitives in modern computer science, underpinning database indexing, signal processing, spatial graphics, and cryptographic hardware. Classical comparison-based sorting paradigms, such as QuickSort, MergeSort, and HeapSort, are fundamentally bound by the theoretical lower bound complexity of $\Omega(N \log N)$. Beyond computational complexity, modern high-performance microarchitectures expose significant hardware-level penalties associated with comparison-based routines. Specifically, conditional branching statements (e.g., `if (A > B)`) trigger instruction pipeline flushes and branch misprediction penalties in deeply pipelined superscalar CPUs, severely degrading instruction-level parallelism (ILP).

To circumvent the comparison bottleneck, non-comparative sorting algorithms such as Counting Sort, Bucket Sort, and Radix Sort achieve linear time complexity $O(N)$ under specific distribution assumptions. However, traditional distribution-based algorithms present severe architectural trade-offs:

- 1) **Memory Footprint Expansion:** Counting Sort exhibits an $O(K)$ spatial overhead proportional to the dynamic range K , making it intractable for full 64-bit domain spaces (2^{64}).
- 2) **Cache Locality Degradation:** Standard Radix Sort requires multiple passes over non-contiguous memory locations, causing high L1/L2 data cache line evictions.
- 3) **Incompatibility with Residue Architectures:** In domain-specific hardware, such as Digital Signal Processors (DSPs), Radar processing units, and Residue Number System (RNS) coprocessors, numbers are naturally represented in modular residues. Converting RNS data back to positional binary notation solely for comparison introduces prohibitive computational overhead.

To address these limitations, this paper introduces *CRT Sparse Sort*, a novel non-comparative sorting framework that maps 1D integer datasets into a two-dimensional residue-coordinate space (S, L) defined by the Chinese Remainder Theorem (CRT) across $k = 9$ coprimary moduli. By exploiting the geometric properties of residue systems, the proposed method converts non-linear dynamic range sorting into deterministic spatial index classification.

1.1 Key Contributions

The primary contributions of this paper are summarized as follows:

- **Formulation of Residue-Coordinate Mapping (S, L) :** We establish a rigorous mathematical proof demonstrating that a 1D sequence under a CRT periodic domain can be uniquely decomposed into a segment index $S(R)$ and a local phase $L(R)$, strictly preserving monotonic algebraic order without requiring pairwise comparison operators.
- **Mirror Symmetry Optimization:** We propose a mid-point reflection mechanism (M_{MID}) that exploits the geometric center of the CRT period. This technique achieves a 50% reduction in residue coordinate storage space while maintaining linear time unmirroring.

- **64-bit Bit-Packing Key & Block Tiling:** We present a composite 64-bit key architecture that integrates the quotient block Q , segment S , and local offset L into a single `uint64_t` representation. This design reduces multi-criteria sorting to a single flat-array Radix Sort pass with minimal stack memory usage (2 KB), achieving $O(1)$ L1 Cache overhead.
- **Branch-Free Hardware-Friendly Execution:** The proposed pipeline eliminates conditional branching inside the inner execution loops. This guarantees deterministic runtime behavior, eliminates side-channel timing attack vectors, and allows direct $O(1)$ clock cycle implementation on parallel hardware accelerators (FPGA/ASIC).

2 RELATED WORK

Sorting algorithms and residue magnitude comparisons have been extensively studied over the past decades. Due to the non-positional nature of the Residue Number System (RNS), magnitude comparison between modular numbers remains an intrinsically difficult operation, traditionally requiring full CRT reconstruction or hardware-heavy Base Extension algorithms.

2.1 Comparison-Based vs. Distribution-Based Sorting

Modern comparison-based sorting implementations, such as Introsort (`std::sort` in C++ STL), combine QuickSort, HeapSort, and Insertion Sort to guarantee $O(N \log N)$ worst-case performance. However, research by Neubert (1998) on FlashSort demonstrated that interpolation-based distribution sorting can achieve $O(N)$ runtime by predicting element locations. Nevertheless, FlashSort relies heavily on uniform data distribution and degrades to $O(N^2)$ when handling skewed datasets. Standard binary Radix Sort operates independently of data distribution, but requires multiple passes (w/d passes for bit-length w and digit-size d), incurring high memory bandwidth penalties.

2.2 Magnitude Comparison in Residue Number Systems

Classical approaches to magnitude comparison in RNS, pioneered by Szabo and Tanaka (1967) and later refined by Wang et al. (2000), predominantly rely on Mixed-Radix Conversion (MRC) or Core Function evaluations. These techniques are tailored for individual pairwise comparison and demand significant computational overhead.

The fundamental departure of *CRT Sparse Sort* from prior literature lies in its strategy: Rather than performing pairwise comparison or global RNS-to-binary conversion, our algorithm introduces a **2D geometric coordinate mapping** (S, L) that linearizes the dataset into unified 64-bit packed keys. This enables batch sorting in $O(N)$ linear time utilizing flat contiguous memory layouts.

3 THEORETICAL FOUNDATION

Consider the set of the first $k = 9$ prime moduli $P = \{p_1, p_2, \dots, p_9\} = \{3, 5, 7, 11, 13, 17, 19, 23, 29\}$. The total CRT product period is defined as:

$$M_{\text{TOTAL}} = \prod_{i=1}^9 p_i = 3,234,846,615 \quad (1)$$

The geometric center of symmetry is determined at:

$$M_{\text{MID}} = \left\lfloor \frac{M_{\text{TOTAL}}}{2} \right\rfloor = 1,617,423,307 \quad (2)$$

Theorem 1 (Order Preservation in Coordinate Space).

For any integer R within the period $0 \leq R < M_{\text{TOTAL}}$, the position of R is uniquely identified by the 2D coordinate pair $(S(R), L(R))$ via two algebraic functions:

$$S(R) = 1 + \sum_{i=1}^9 \left\lfloor \frac{R}{p_i} \right\rfloor \quad (3)$$

$$L(R) = R - \max_{i=1 \dots 9} \left(p_i \cdot \left\lfloor \frac{R}{p_i} \right\rfloor \right) \quad (4)$$

Proof. The floor function $\lfloor R/p_i \rfloor$ is a monotonically non-decreasing step function with respect to R . Consequently, $S(R)$ is a monotonically increasing function $S(R_1) \leq S(R_2)$ for all $R_1 < R_2$. Within the same segment $S(R_1) = S(R_2)$, no prime moduli boundary is crossed, rendering the boundary term $B = \max(p_i \cdot \lfloor R/p_i \rfloor)$ constant. Under this condition, $L(R) = R - B$ increases linearly with R . Thus, the coordinate pair $(S(R), L(R))$ preserves 100% of the algebraic ordering. ■

4 PROPOSED METHODOLOGY & 6-STEP PIPELINE

4.1 Step 1: 64-bit Block Tiling

To process 64-bit integers exceeding the period M_{TOTAL} , data X is decomposed into:

$$X = Q \cdot M_{\text{TOTAL}} + R \quad (5)$$

where $Q = \lfloor X/M_{\text{TOTAL}} \rfloor$ represents the block quotient and $R = X \pmod{M_{\text{TOTAL}}}$ is the normalized residue.

4.2 Step 2: Mirror Symmetry Split

The residue R is classified against the midpoint M_{MID} :

- If $R \leq M_{\text{MID}}$: Retain R , set `is_upper` = false.
- If $R > M_{\text{MID}}$: Reflect $R_{\text{mirror}} = M_{\text{TOTAL}} - 1 - R$, set `is_upper` = true.

4.3 Step 3: Residue Coordinate Mapping (S, L)

Compute S and L using unrolled fast division loops:

$$q_i = \left\lfloor \frac{R}{p_i} \right\rfloor, \quad S = 1 + \sum_{i=1}^9 q_i, \quad L = R - \max_{i=1 \dots 9} (p_i \cdot q_i) \quad (6)$$

4.4 Step 4: 64-bit Bit-Packing Key

Since $L(R) < 29$, L strictly requires only 5 bits ($2^5 = 32$). The triplet (Q, S, L) is packed into a single `uint64_t` variable:

$$\text{Key} = (Q_{\text{key}} \ll 36) \mid (S \ll 5) \mid (L \ \& \ 0x1F) \quad (7)$$

where $Q_{\text{key}} = Q$ for `lower_group` and $Q_{\text{key}} = \text{MAX_Q} - Q$ for `upper_group`.

4.5 Step 5: Flat Array 8-pass Radix Sort

Execute an 8-bit Radix Sort on `packed_key` independently for `lower_group` and `upper_group`. The histogram array `count[257]` consumes only 2 KB of Stack memory.

4.6 Step 6: Pointer Merge & Unmirroring

Traverse `lower_group` forward and

5 ALGORITHM PSEUDOCODE

Algorithm 1 CRT Sparse Sort $O(N)$

Require: Array A containing N 64-bit integers

Ensure: Sorted array A in ascending order

```

1:  $lower\_group \leftarrow$  Empty List
2:  $upper\_group \leftarrow$  Empty List
3: for each  $x \in A$  do
4:    $Q \leftarrow \lfloor x/M_{TOTAL} \rfloor$ ,  $R \leftarrow x \pmod{M_{TOTAL}}$ 
5:   if  $R \leq M_{MID}$  then
6:      $(S, L) \leftarrow \text{COMPUTE\_COORDINATES}(R)$ 
7:      $key \leftarrow (Q \ll 36) \mid (S \ll 5) \mid (L \ \& \ 0x1F)$ 
8:      $lower\_group.APPEND(x, key)$ 
9:   else
10:     $R_{mirror} \leftarrow M_{TOTAL} - 1 - R$ 
11:     $(S, L) \leftarrow \text{COMPUTE\_COORDINATES}(R_{mirror})$ 
12:     $Q_{key} \leftarrow MAX\_Q - Q$ 
13:     $key \leftarrow (Q_{key} \ll 36) \mid (S \ll 5) \mid (L \ \& \ 0x1F)$ 
14:     $upper\_group.APPEND(x, key)$ 
15:   end if
16: end for
17:  $RADIX\_SORT\_64BIT(lower\_group)$ 
18:  $RADIX\_SORT\_64BIT(upper\_group)$ 
19: return  $MERGE\_POINTERS(lower\_group, upper\_group)$ 

```

6 PERFORMANCE ANALYSIS & EVALUATION

6.1 Theoretical Complexity

- **Time Complexity:** $O(k \cdot N)$ where $k = 9$ is constant. The linear runtime is completely independent of initial data entropy or array pre-sortedness.
- **Space Complexity:** $O(N)$ auxiliary space allocated on contiguous flat structures at 16 bytes/struct.

6.2 Hardware Advantages & Domain-Specific Fit

- 1) **FPGA/ASIC Acceleration:** Allows direct deployment of 9 parallel execution pipelines, evaluating (S, L) in a single clock cycle ($O(1)$).
- 2) **Digital Signal Processing (DSP & Radar):** Directly sorts RNS residue data without requiring reverse $RNS \rightarrow \text{Binary}$ conversions.
- 3) **Deterministic Execution (Branch-Free):** Eliminates control-flow branches in the core sorting loop, mitigating side-channel timing vulnerability risks.

7 CONCLUSION

This paper presented *CRT Sparse Sort*, a non-comparative sorting framework based on CRT residue-coordinate mapping. Combining block tiling, mirror symmetry, and 64-bit key packing, the algorithm demonstrates rigorous mathematical correctness and presents promising potential for designing dedicated hardware sorting accelerators on FPGA/ASIC platforms.